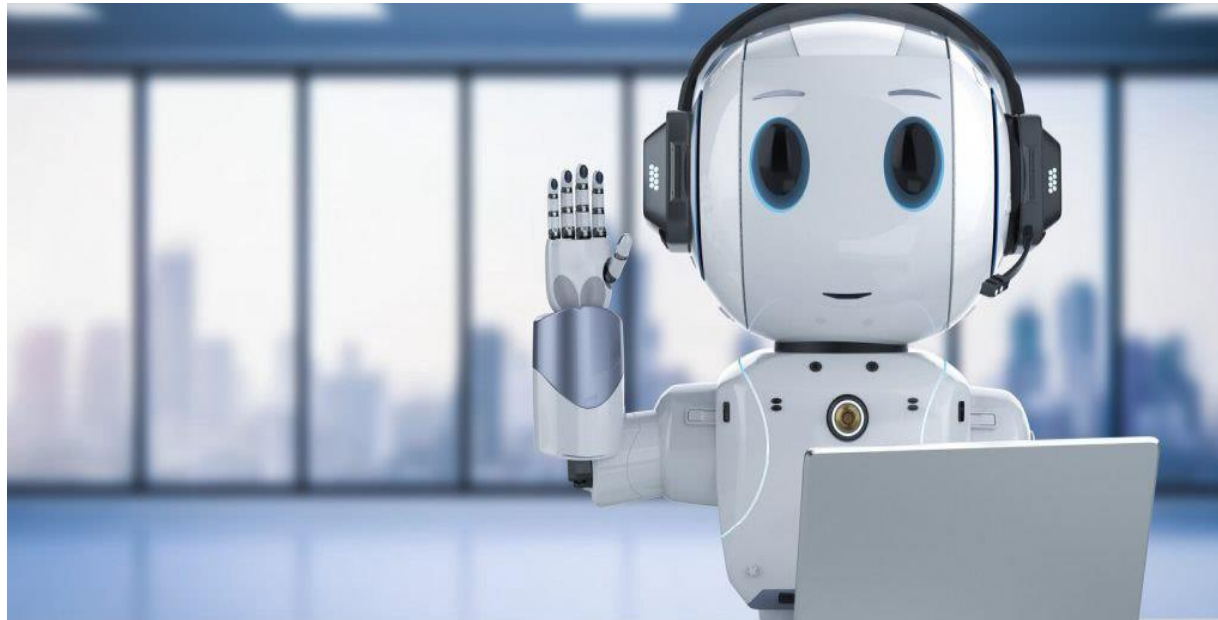


IA para la Educación con TensorFlow en Google Colab



Antes de iniciar...conozcan a mi asistente Mike

<https://asistentemike.streamlit.app/>



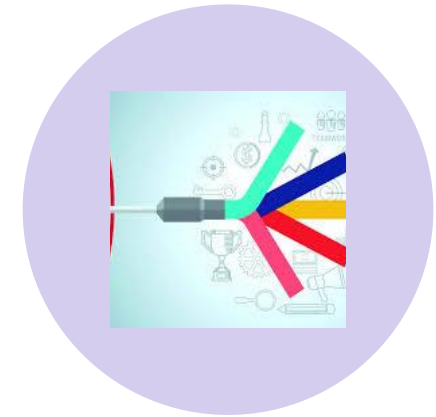
Aspectos generales del curso



ALCANCE: INTRODUCIR A LOS DOCENTES EN EL DISEÑO DE HERRAMIENTAS DE INTELIGENCIA ARTIFICIAL PARA RESOLVER PROBLEMAS EDUCATIVOS A TRAVÉS DE APLICACIONES PRÁCTICAS DESARROLLADAS CON TENSORFLOW EN PYTHON



OBJETIVO GENERAL: INTRODUCIR A LOS DOCENTES EN LA CREACIÓN DE MODELOS SIMPLES DE APRENDIZAJE AUTOMÁTICO QUE PERMITAN OPTIMIZAR EL PROCESO EDUCATIVO.



OBJETIVOS ESPECÍFICOS:

- CREACIÓN DE APPS EDUCATIVAS CON IA.
- DESARROLLAR Y ENTRENAR MODELOS SIMPLES DE APRENDIZAJE AUTOMÁTICO.
- FOMENTAR EL USO DE LA IA PARA MEJORAR PROCESOS EDUCATIVOS.

Contenido del curso



Módulo 1. Introducción a TensorFlow y su impacto en la IA y en la Educación



Módulo 2. Google Colab como entorno de desarrollo



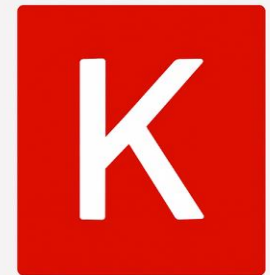
Módulo 3. Primeros pasos en Google Colab con TensorFlow



Módulo 4. Introducción a las redes neuronales con Keras



Módulo 5. Actividades prácticas



Encuestas de diagnóstico

Encuesta 1. Nivel de manejo de Google Colab

<https://www.menti.com/aljjh94vgjhh>

Código: 4853 6482

Objetivo: Conocer la experiencia previa de los participantes en cuanto al uso y manejo del entorno de trabajo, con el fin de adaptar, en caso de ser necesario, algunos contenidos del curso.



Encuesta 2. Nivel de manejo de Python y TensorFlow

<https://www.menti.com/alv1g81rxu3h>

Código: 7228 8500

Objetivo: Identificar el nivel de experiencia y conocimientos de los participantes sobre el lenguaje de programación Python y la librería TensorFlow, con el fin de ajustar, de ser necesario, las actividades prácticas diseñadas.



Encuesta 3. Expectativas generales del curso

<https://www.menti.com/al3ihdixft2a>

Código: 4155 3227

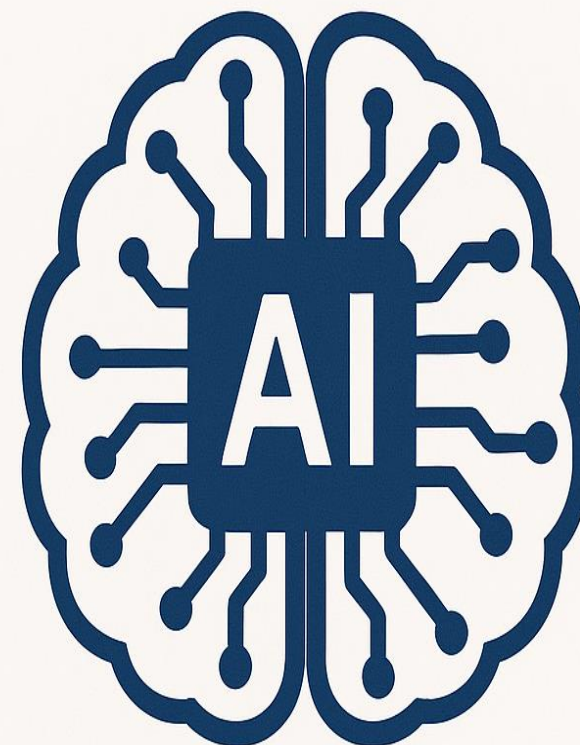
Objetivo: Comprender las expectativas e intereses de los participantes con el fin de personalizar el aprendizaje.



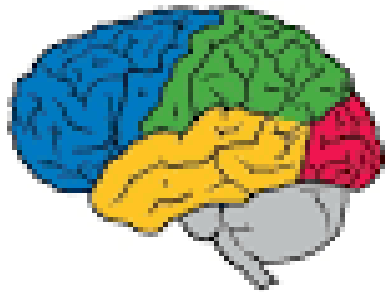
Módulo 1

Se abordará lo siguiente:

- Breve historia del DL en Google
- ¿Qué es y para qué sirve TensorFlow?
- Impacto en la educación
- Problemas educativos que se pueden abordar con TensorFlow



Breve historia del Deep Learning en Google



Large-Scale Deep Learning With TensorFlow

Jeff Dean
Google Brain team
g.co/brain

In collaboration with **many** other people at Google

- Ha sido usada en diversos proyectos de redes neuronales profundas
- Tuvo errores de diseño que limitaron su uso y flexibilidad.
- Google comenzó a trabajar en su sucesor y se convirtió en TensorFlow, el cual se lanzó al público a finales del 2015.

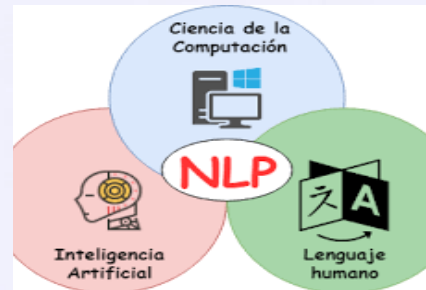
Un poco sobre TensorFlow

¿Qué es?



Es una biblioteca de Python de código abierto desarrollada por Google y diseñado para modelos de ML y DL

¿Para qué sirve?



Visión por computadoras
NLP
Apps de IA
Chatbots

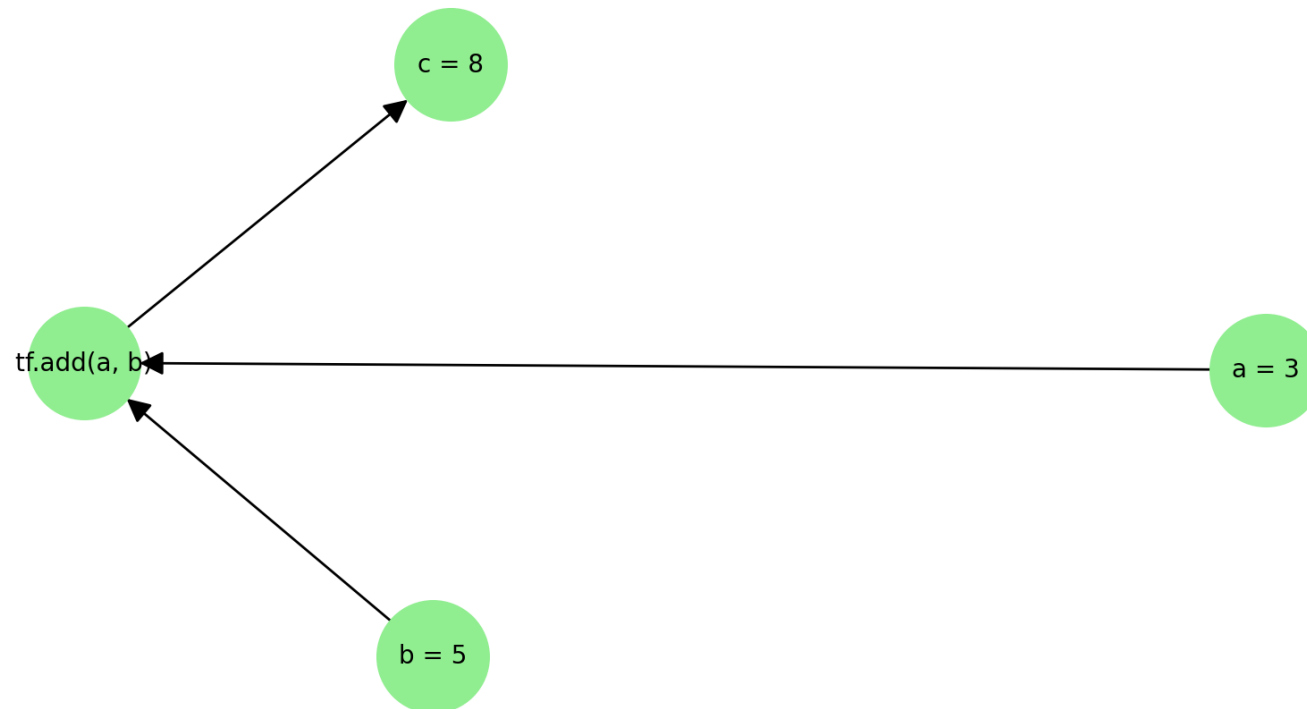
Algunos Datasets

```
rcpy
"C:/Lessons/PythonDesc/DC.gd
v.workspace = mypath
rcpy.da.Describe("boundary")
baseName: {desc["baseName"]}
extension: {desc["extension"]}
dataType: {desc["dataType"]}
eType" in desc:
t(f'shapeType: {desc["shapeTy
v in desc.items():
int(f"(k): {v}")
```

MNIST
CIFAR-10
IMDB
WIKI40B

Un poco sobre TensorFlow

Grafo computacional simple en TensorFlow



Se llama TensorFlow porque está diseñado para trabajar con flujo de tensores (datos).

TensorFlow = Flujo + Datos en forma de tensores

Flow (flujo) se refiere a cómo los datos (tensores) fluyen de forma organizada por un grafo computacional (como si fuera una tubería matemática).

La estructura del grafo computacional es la siguiente:

- *Nodos (círculos)*: Cada nodo representa una operación matemática, como suma, multiplicación, activación, convolución, etc.
- *Aristas o conexiones (flechas)*: Cada conexión transporta un tensor, es decir, los datos de entrada o salida de cada operación.

Impacto de TensorFlow en la Educación

- Fomenta el pensamiento computacional y analítico en los estudiantes y docentes.
- Facilita el desarrollo de tecnologías accesibles e inclusivas para personas con capacidades diferentes.
- Aumenta la eficiencia de los procesos administrativos.
- Promueve la educación basada en evidencia mediante el análisis masivo de datos educativos (Learning Analytics)
- Permite automatizar diversos tipos de procesos y tareas que se desarrollan en el entorno educativo.

Algunos problemas educativos que se pueden resolver con TensorFlow

- Deserción escolar: predicción temprana basada en patrones de rendimiento y comportamiento.
- Evaluación automática: calificación y retroalimentación de tareas.
- Diagnóstico de dificultades de aprendizaje.
- Educación personalizada: recomendación de contenidos de acuerdo al perfil de aprendizaje.
- Inclusión educativa: diseño de apps para personas con capacidades diferentes.
- Detección de plagio: análisis de similitud semántica en textos.
- Análisis emocional: identificación de emociones.
- Optimización del diseño instruccional.

Módulo 2

Se abordará lo siguiente:

- Introducción al entorno de Google Colab
- Elementos básicos del entorno
- Ventajas de usar Google Colab en la educación



Introducción a Google Colab

Es un entorno de programación gratuito basado en Jupyter Notebook y el cual se ejecuta en los servidores de la nube de Google. Esta herramienta fue diseñada principalmente para proyectos de Machine Learning, enseñanza de la programación en Python, colaboración académica en tiempo real, creación de cuadernos de trabajo interactivos y además permite:

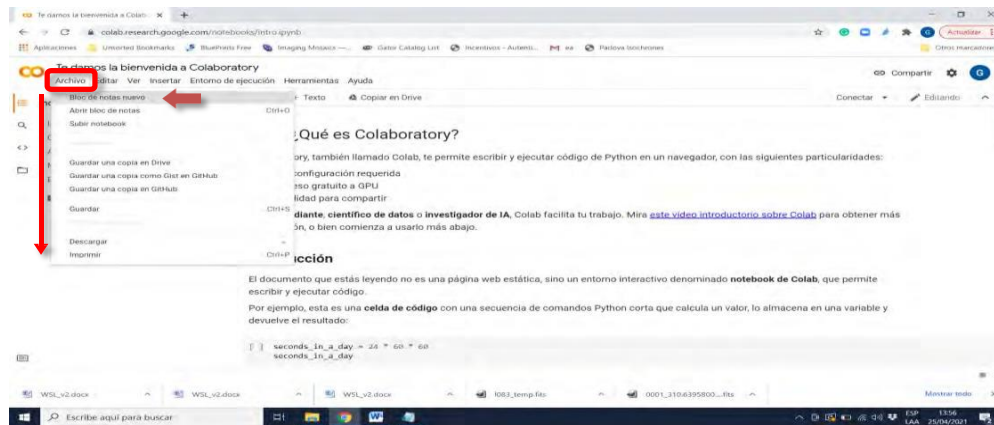
Escribir, ejecutar y compartir códigos de Python (sin instalación ni configuración del software).

Ejecutar notebooks en la nube de forma gratuita.

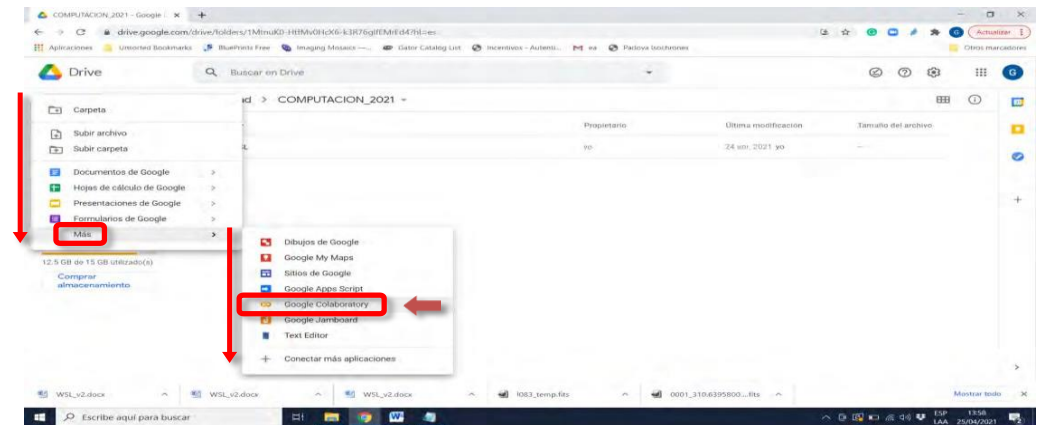
Usar recursos computacionales como GPU y TPU.

Acceder fácilmente a archivos de Google Drive.

Acceso al entorno

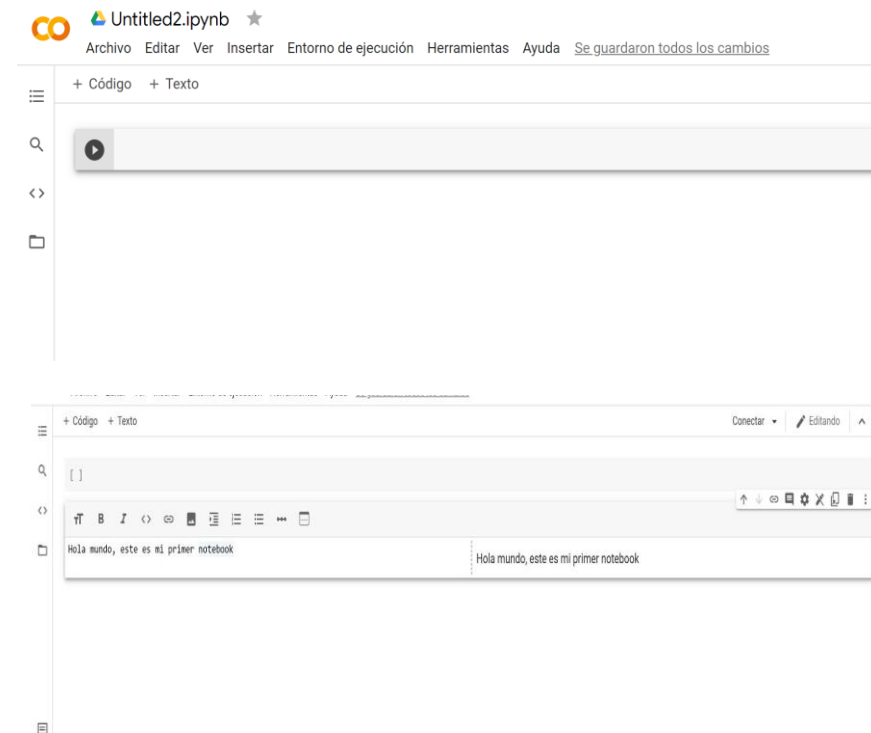
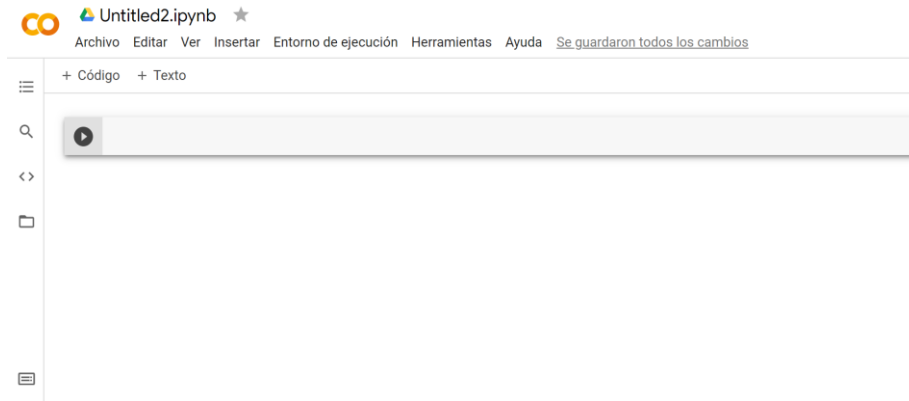


Ir al sitio de “Google Colab”
(<https://colab.research.google.com/>), ir a “Archivo” (arriba a la izquierda) y elegir la opción “Block de notas nuevo”

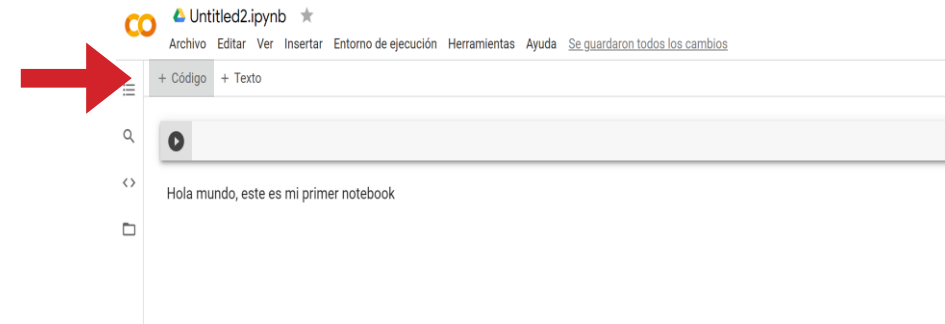
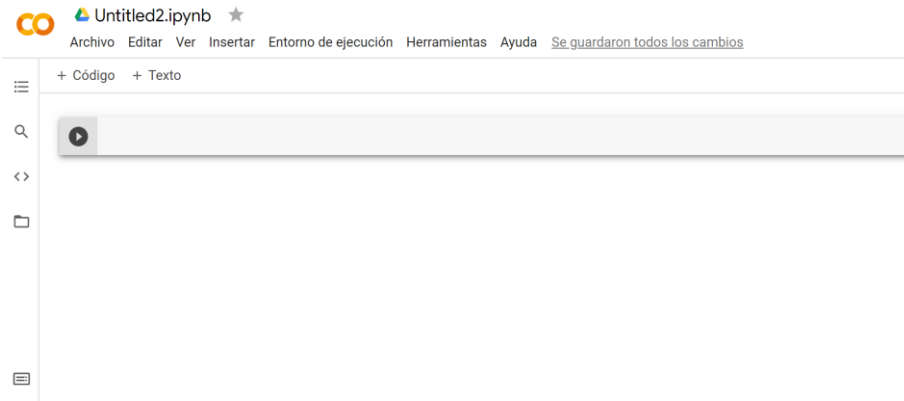


Ir al sitio de su Drive de Google personal o institucional

Jupyter Notebook (celdas de texto)



Jupyter Notebook (celdas de código)



Jupyter Notebook (combinación de celdas)

The screenshot displays a Jupyter Notebook interface with three code cells. Each cell has a toolbar with formatting options (bold, italic, monospace, etc.) and a right-hand sidebar with a title and a list of items.

- Cell 1: Carga de paquetes**
 - Code:

```
##Carga de paquetes##  
- Numpy  
- Matplotlib  
  
[ ] import numpy as np  
import matplotlib.pyplot as plt
```
 - Right sidebar: **Carga de paquetes**
 - Numpy
 - Matplotlib
- Cell 2: Estadística básica**
 - Code:

```
##Estadística básica##  
- Definición de un conjunto de números al azar con distribución  
uniforme \\  
$ {x_1}, x_2, \dots, x_N$  
- Valor medio o promedio \\  
$ prom = \frac{\sum_{i=1}^N x_i}{N} = \frac{x_1 + x_2 + \dots + x_N}{N}$  
- Desviación estándar \\  
$ stdev = \sqrt{\frac{\sum_{i=1}^N (x_i - prom)^2}{N-1}}$
```
 - Right sidebar: **Estadística básica**
 - Definición de un conjunto de números al azar con distribución uniforme
 $x_1, x_2, \dots, x_N$
 - Valor medio o promedio
$$prom = \frac{\sum_{i=1}^N x_i}{N} = \frac{x_1 + x_2 + \dots + x_N}{N}$$
 - Desviación estándar
$$stdev = \sqrt{\frac{\sum_{i=1}^N (x_i - prom)^2}{N-1}}$$
- Cell 3: Gráfico con Matplotlib**
 - Code:

```
##Gráfico con Matplotlib##  
Histograma de los datos junto con su valor medio y desviación  
estándar  
  
[ ] # Estadística básica con Numpy  
# -----  
# Conjunto de N valores con distribución uniforme entre 0 y A  
N = 1000  
A = 50  
x = A * np.random.rand(N)  
  
# Cálculos estadísticos  
prom = np.mean(x)  
stdev = np.std(x)  
x_min = np.min(x)  
x_max = np.max(x)  
  
[ ] # Salida de resultados  
# -----  
print ('-----')  
print ('N =', N)  
print ('promedio =', prom)  
print ('desviación estándar =', stdev)  
print ('valor mínimo =', x_min)  
print ('valor máximo =', x_max)  
print ('-----')
```
 - Right sidebar: **Gráfico con Matplotlib**
 - Histograma de los datos junto con su valor medio y desviación estándar

Ventajas de su uso

Ventaja	Descripción
Gratuito y accesible	Solo se necesita una cuenta de Google
Sin instalaciones	Ideal para entornos educativos con limitaciones técnicas
Recursos potentes	Ofrece GPU y TPU, útiles para la IA
Facilidad de compartir	Los notebooks se comparte como archivos de Google Docs
Colaboración en tiempo real	Permite que varios usuarios trabajen simultáneamente
Soporte para bibliotecas modernas	TensorFlow, Pytorch, Keras, Pandas, NumPy, etc, ya vienen preinstaladas

Empecemos a trabajar...Actividad 1

- El **objetivo de la actividad** consiste en clasificar actividades educativas en automatizables o no automatizables con IA.

Antes de iniciar la actividad tome en cuenta que La diferencia entre actividades automatizables y no automatizables con IA radica principalmente en el tipo de habilidades humanas necesarias para realizarlas y en la naturaleza de los datos disponibles.

- Las actividades automatizables son aquellas que:
 - Tienen reglas claras o patrones repetitivos.
 - Pueden resolverse con datos estructurados.
 - Requieren procesamiento lógico o estadístico, no juicio humano complejo.
 - Se pueden entrenar con ejemplos previos.

Las actividades no automatizables son aquellas que:

- Requieren juicio ético, empatía o interpretación compleja.
- Implican contexto cultural, emocional o personal.
- Tienen alta variabilidad y pocos datos claros para entrenar un modelo.

Empecemos a trabajar...Actividad 1

¿Por qué es importante conocer qué tipo de actividades son automatizables con IA y cuáles no?

- ***Evita malgastar tiempo y recursos***, ya que si no se distingue desde un inicio se puede perder tiempo intentando resolver problemas que requieren mucha participación humana.

- ***Saber qué automatizar te guía hacia el enfoque y el modelo adecuado***, por ejemplo:

Análisis de datos de rendimiento → redes neuronales simples.

Para reconocimiento de voz → redes convolucionales o recurrentes.

Para predicción de abandono escolar → modelos supervisados

- ***Tener claro los objetivos educativos***, con el fin de responder ¿qué problema educativo pretendo resolver? ¿qué datos necesito? ¿qué decisiones tomará la IA y cuáles no?

Empecemos a trabajar...Actividad 1

Objetivo: Identificar oportunidades de aplicación de IA en el contexto educativo.

Alcance: Reflexionar sobre actividades comunes en clase y decidir cuáles podrían automatizarse.

Instrucciones:

En equipos de 5 participantes, discutan y enlisten 10 actividades frecuentes en su práctica docente. Clasifíquenlas en automatizable con IA, no automatizable o parcialmente automatizable y justifiquen cada clasificación en una tabla en Google Docs.

# de Actividad	Nombre o descripción breve de la actividad	¿Automatizable con IA? (Sí, No, Parcialmente)	Breve justificación
1	Calificación de exámenes de opción múltiple	Sí	La IA puede corregir automáticamente mediante claves predefinidas, reduciendo tiempo y errores humanos
2	Explicación personalizada de conceptos complejos	Parcialmente	La IA puede ofrecer explicaciones básicas con chatbots o asistentes, pero no reemplaza del todo la interacción humana personalizada
3	Facilitación de debates en clase	No	La moderación de debates requiere habilidades sociales, gestión de emociones y adaptación que la IA aún no domina

Módulo 3

Se abordará lo siguiente:

- Ejercicios básicos desarrollados directamente en el entorno de Google Colab usando Python y algunas de sus librerías como TensorFlow
- Accesar al repositorio de Google Drive del curso

[Drive CADI IA TensorFlow](#)



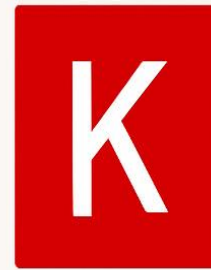
Ejercicios básicos en Google Colab y TensorFlow

- Cálculo de la suma de los n primeros enteros positivos.
- Cálculo del índice de masa corporal, se almacene en una variable y se muestre el resultado.
- Creación de tensores simples (tensores escalares) y operaciones básicas.
- Creación de tensores multidimensionales (matrices).
- Cálculo del promedio de calificaciones
- Visualización del gráfico de una función matemática
- Creación de un tensor con valores del 0 al 9 y elevarlo al cuadrado.
- Cálculo de la media, varianza y desviación estándar de un conjunto de datos.
- Creación de una función definida por el usuario y su gráfico.
- Uso de ciclo for en TensorFlow (cálculo de una suma acumulada).
- Entrenamiento básico de modelo de regresión lineal.
- Clasificación básica usando `tf.where`

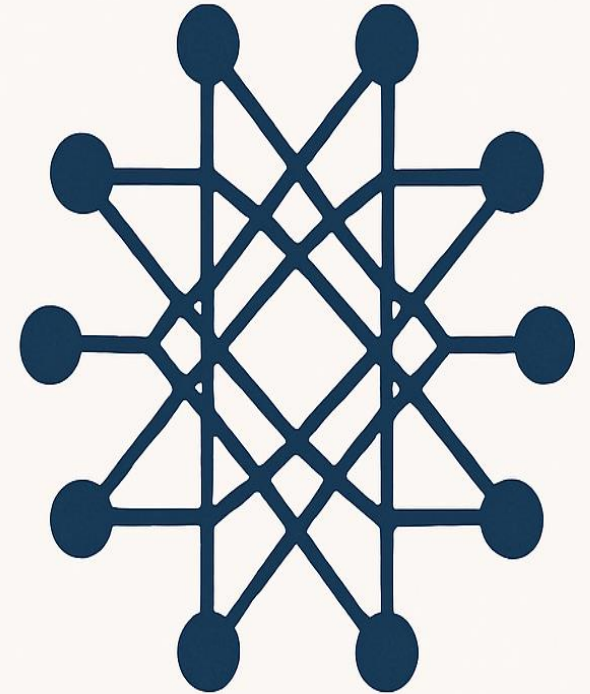
Módulo 4

Se abordará lo siguiente:

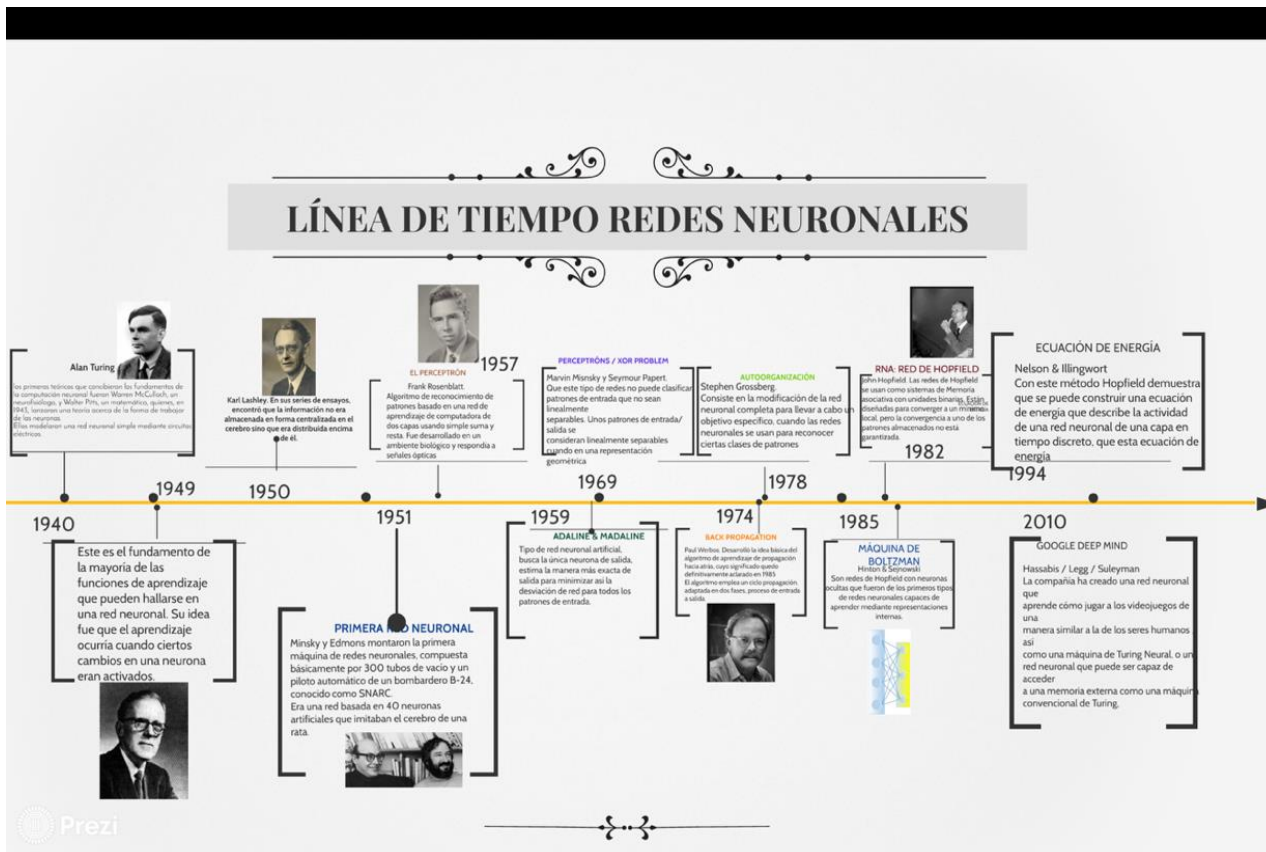
- Introducción a las redes neuronales. Capas y tipos de aprendizajes.
- Comparación del funcionamiento de una neurona biológica vs una neurona artificial.
- Introducción a la biblioteca Keras



Keras



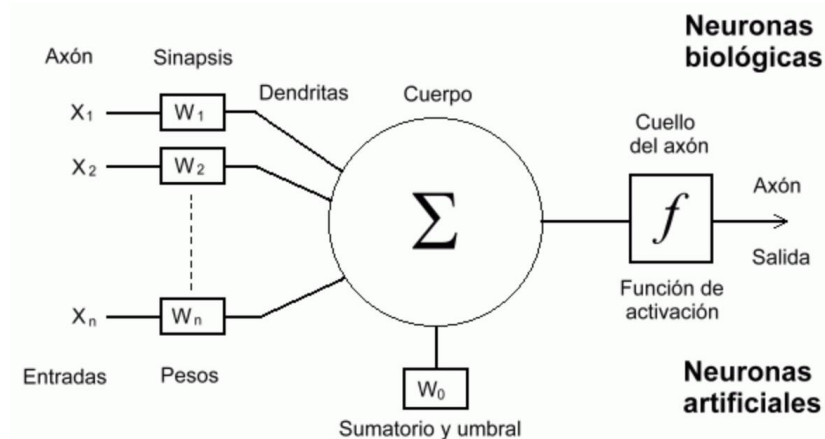
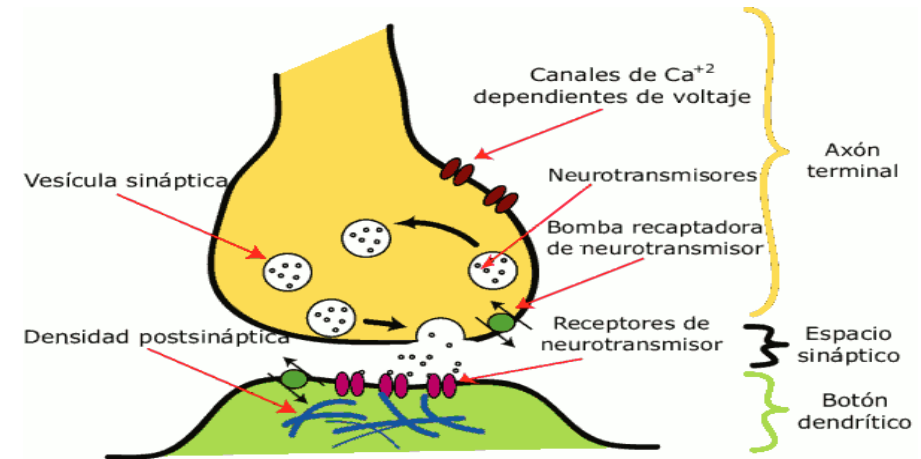
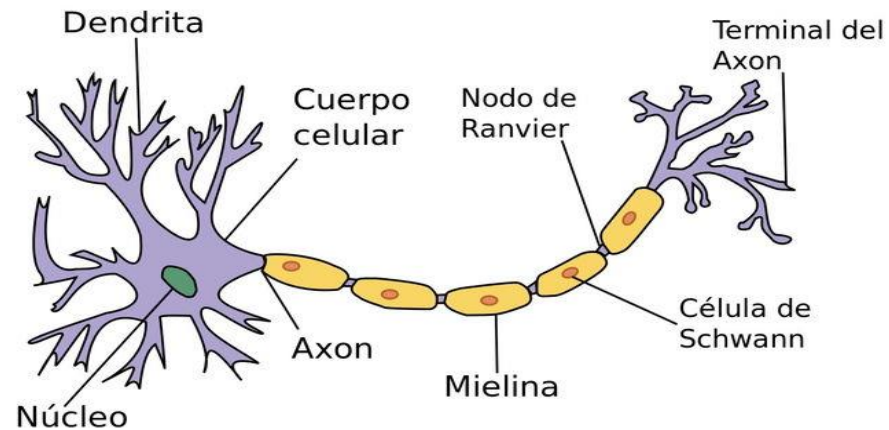
Un poco de historia (redes neuronales)



- A principios de los años 40, primeros modelos por Warren McCulloch y Walter Pitts.
- En 1959, Bernard Widrow y Marcian Hoff desarrollaron los algoritmos Adaline y Madaline.
- A finales de los años 40, Donald Hebb definió dos conceptos muy importantes para el proceso de aprendizaje.
- En los años 80, el interés por las redes neuronales se renovó. En 1984, Kohonen desarrolló una familia de redes de memoria asociativa. En 1985, Paul Werbos desarrolló el algoritmo Backpropagation. En 1986, Hinton y Sejnowski desarrollaron la máquina de Boltzmann.
- A finales de los años 50, Frank Rosenblatt introdujo el Perceptrón que dio lugar a la regla de aprendizaje Delta.

La neurona biológica vs neurona artificial

(funcionamiento)



La neurona biológica vs neurona artificial (funcionamiento)

Entradas ($x_1, x_2, \dots$): estos valores pueden ser enteros, reales o binarios y equivaldrían a los impulsos que envían otras neuronas a través de sus dendritas.

Pesos ($w_1, w_2, \dots$): equivaldrían a los mecanismos de sinapsis necesarios para transmitir el impulso.

El producto $x_i w_i$ equivaldría a las señales químicas inhibitoras y excitadoras que se dan en la neurona. Estos valores son la entrada de la función de activación, que convierte todo el conjunto de valores en uno solo llamado potencial. La función de ponderación suele ser la suma ponderada de las entradas y los pesos sinápticos.

La salida de la función de ponderación llega a la función de activación que transforma este valor en otro en el dominio en que trabajen las salidas de las neuronas. Suele ser una función no lineal como la función paso o la función sigmoide, aunque también se pueden usar funciones lineales.

En resumen, podemos definir las siguientes analogías:

- Conexiones sinápticas $\leftrightarrow$ Conexiones ponderadas
- Efectividad de las sinapsis $\leftrightarrow$ Peso de las conexiones
- Efecto excitador o inhibitor de una conexión $\leftrightarrow$ Signo del peso de una conexión
- Activación $\rightarrow$ Tasa de disparo $\leftrightarrow$ Función de activación $\rightarrow$ Salida

Elementos básicos de una Red Neuronal Artificial

Función de red o propagación:

Esta función se encarga de transformar las diferentes entradas que provienen de la capa anterior en el potencial de la neurona actual. Permite combinar las entradas simples en una sola entrada tomando en cuenta dichas entradas y los respectivos pesos.

$$input = (x_1:w_1) * (x_2:w_2) * \dots * (x_n:w_n)$$

Este operador se suele representar como la suma ponderada de todas las entradas simples multiplicadas por los pesos correspondientes.

$$y' = \sum_{i=1}^n x_i w_i$$

Función de activación:

Esta función combina el potencial que tiene la función de propagación con el estado actual de la neurona para conseguir el estado futura de ésta (activada o desactivada).

Entre las más usadas están las siguientes:

- ✓ Función identidad
- ✓ Función escalón binario
- ✓ Función sigmoide
- ✓ Función tangente hiperbólica
- ✓ Función rectificadora (ReLU – Rectified Linear Unit)
- ✓ Función Softplus

Elementos básicos de una Red Neuronal Artificial

Función de pérdida:

La función de pérdida es uno de los parámetros necesarios para compilar un modelo.

Algunas de las funciones más conocidas que nos proporciona la biblioteca Keras son:

- ✓ `mean_squared_error (y_true, y_pred)`: Calcula el error cuadrático medio.
- ✓ `mean_absolute_error / mae`: Calcula el error medio absoluto.
- ✓ `binary_crossentropy`: Calcula la entropía cruzada en problemas de clasificación binaria. También es conocida como pérdida logarítmica.

Regularizadores:

Los regularizadores permiten aplicar penalizaciones en las capas durante el proceso de optimización. Estas penalizaciones se incorporan a la función de pérdida que la red optimiza con el fin de evitar que la red neuronal aprenda “demasiado bien” los datos de entrenamiento; es decir, para prevenir el sobreajuste (overfitting). En otras palabras, agregan un “castigo o penalización” al valor de la función de pérdida si los pesos del modelo crecen demasiado o si se vuelven muy complejos.

Por ejemplo:

```
model.add(Dense(64, input_dim=64,  
W_regularizer=12 (0.01),  
activity_regularizer=activity_12(0.01)))
```

Elementos básicos de una Red Neuronal Artificial

Optimizadores:

El optimizador es otro de los valores necesarios para compilar un modelo (junto con la función de pérdida). Ajustan los pesos de la red neuronal durante el entrenamiento para que la función de pérdida se minimice lo más rápido y eficientemente posible.

SGD: aproximación estocástica del método de gradiente descendiente.

Adagrad: algoritmo basado en gradiente que adapta el ratio de aprendizaje a los parámetros, realizando grandes actualizaciones cuando éstos son infrecuentes, y pequeñas actualizaciones cuando éstos son frecuentes.

Adadelta: es una extensión del algoritmo Adagrad.

Adam: algoritmo basado en el gradiente de primer orden de funciones estocásticas objetivas. Es un método sencillo de aplicar, es eficiente computacionalmente, necesita poca capacidad de memoria y es muy usado en problemas con grandes cantidades de datos y/o parámetros.

Callbacks:

Son funciones que pueden ser aplicadas en ciertos momentos del proceso de entrenamiento de la red neuronal. Estas funciones permiten guardar el modelo cuando mejora, detener el entrenamiento si no mejora (EarlyStopping), ajustar la tasa de aprendizaje automáticamente y registrar métricas para visualización (como TensorBoard).

`BaseLogger()`: se aplica en todo modelo de Keras y acumula las medias de las métricas que se han definido para ser monitoreadas por época.

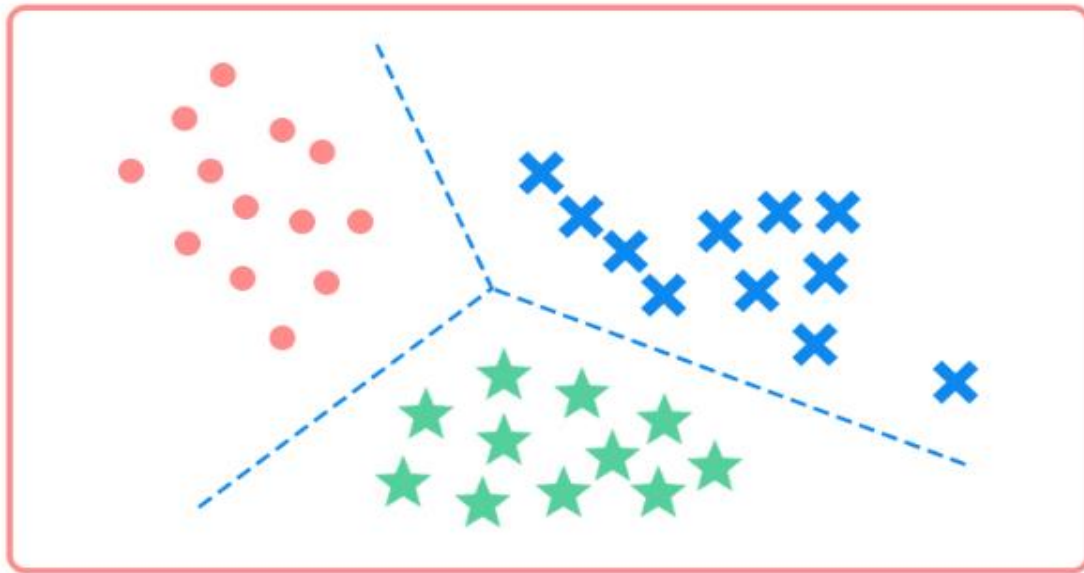
`Callback()`: clase base abstracta utilizada para crear nuevos callbacks. Como parámetros, recibe los propios para configurar el entrenamiento (número de ciclos, verbosidad, ...), así como la instancia del modelo que se quiere entrenar.

`ProgbarLogger()`: Callback que muestra métricas en la salida estándar (stdout).

`ModelCheckpoint(...)`: almacena el modelo después de cada época. Como argumento recibe el fichero donde deja la información, la cantidad de elementos a monitorear, el modo de verbosidad, un booleano para decidir si sobrescribir o no el mejor modelo, el modo, y otro booleano para guardar solo pesos o información de todo el modelo.

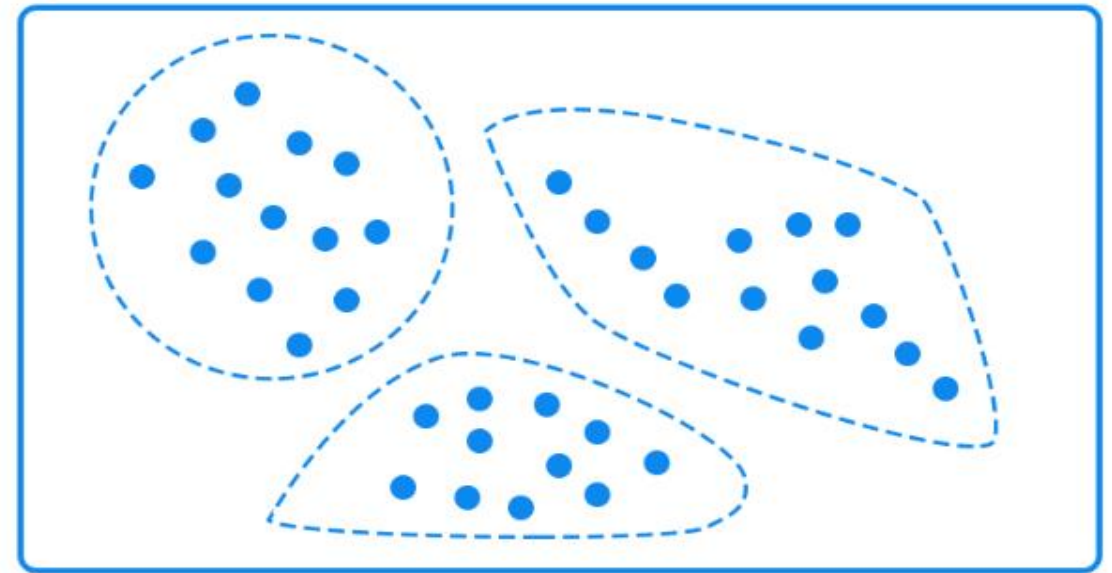
Tipos de aprendizajes en redes neuronales

Classification



Supervised learning

Clustering



Unsupervised learning

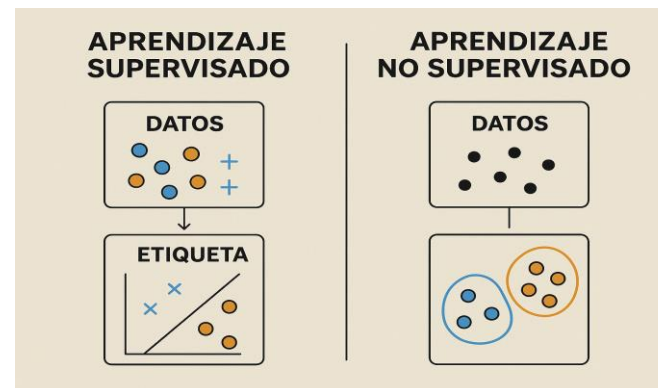
Diferencias entre Aprendizaje Supervisado y No Supervisado

Modelos de Aprendizaje Supervisado

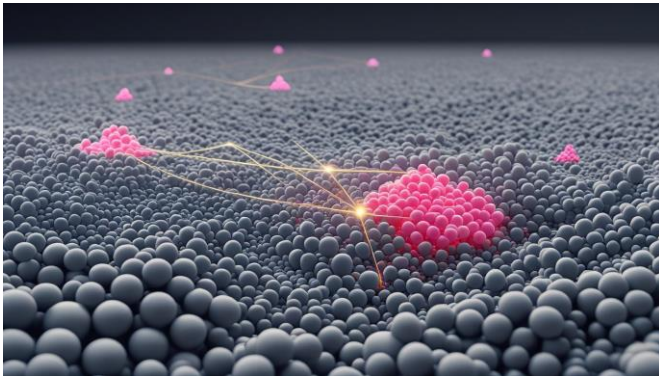
- ✓ Los datos de entrenamiento son etiquetados previamente por el humano.
- ✓ El algoritmo usa datos etiquetados con el fin de aportar contexto y proporcionar los valores de salida deseados para que el modelo de la respuesta correcta.
- ✓ Se centran en aprender las relaciones entre los datos de entrada y las salidas.

Modelos de Aprendizaje No Supervisados

- ✓ No usa datos de entrenamiento etiquetados.
- ✓ El algoritmo funciona de forma independiente para aprender la estructura de los datos, es decir, identifica el patrón natural de los datos.
- ✓ Se centra en descubrir patrones en datos sin procesar ni etiquetar.



Otros modelos de aprendizajes



Aprendizaje semisupervisado

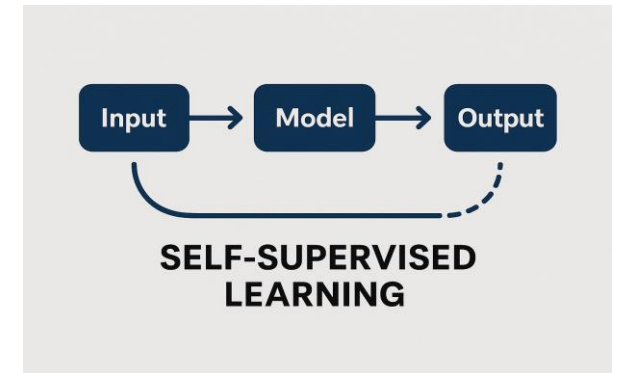
Sólo son etiquetados por el humano una parte de los datos

MODELO DE APRENDIZAJE POR REFUERZO



Aprendizaje por refuerzo

Imita el proceso de aprendizaje por ensayo y error que los humanos usan para lograr sus objetivos



Aprendizaje autosupervisado

Las etiquetas de los datos son generados por el propio algoritmo a partir de datos no estructurados.

Un poco sobre la biblioteca Keras

Es una biblioteca de código abierto y de alto nivel para el desarrollo y el entrenamiento de modelos de aprendizaje automático y de aprendizaje profundo. Fue desarrollada por François Chollet y es parte del ecosistema de TensorFlow. Algunas de sus características que la hace ideal son:

Tiene una API simple y fácil de usar.

Es rápida y eficiente para el desarrollo de modelos de aprendizaje automáticos complejos.

Es compatible con diferentes frameworks, como TensorFlow, Pytorch y JAX.

Soporta redes neuronales tanto recurrentes como convolucionales, incluso combinación de ambas.

Un poco más sobre Keras

Keras se basa en un concepto llamado “capas”, las cuales constituyen los bloques fundamentales para construir modelos de aprendizaje automático.

Algunas de las capas más utilizadas con Keras son las siguientes:

- **Capas de convolución**, las cuales son usadas para procesar imágenes y otros tipos de datos con representaciones en 2D.
- **Capas de pooling**, las cuales son usadas para reducir la dimensionalidad de los datos.
- **Capas densas**, las cuales son las más comunes en la construcción de modelos de aprendizaje automático, aprende las relaciones entre los datos para poder llevar a cabo la tarea objetivo.
- **Capas LSTM (Long Short-Term Memory)**, las cuales son usadas para procesar datos secuenciales, como series temporales y textos.
- **Capas dropout**, las cuales son usadas para regularizar el modelo y evitar sobreajuste, es decir, se eliminan un conjunto de neuronas de manera aleatoria durante el entrenamiento.
- **Capas de embedding**, las cuales son usadas para convertir datos en representaciones numéricas densas, lo que permite que el modelo aprenda relaciones entre los datos y generalmente se utilizan en tareas de procesamiento de lenguaje natural (NLP).

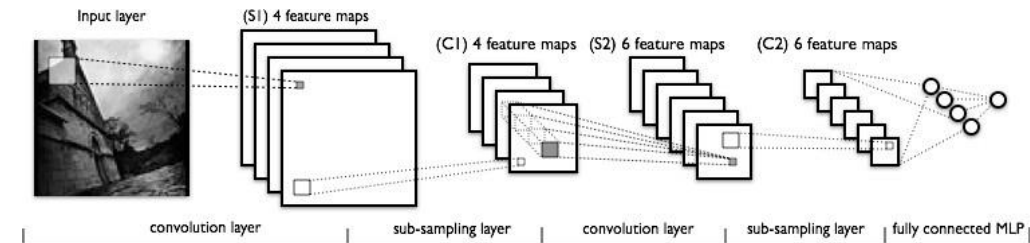
Un poco sobre la estructura de las Redes Neuronales Convolucionales

Características principales:

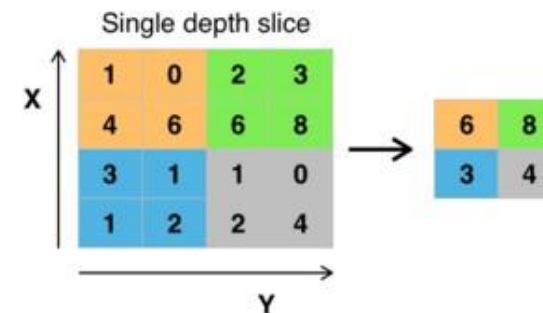
Se usan principalmente para resolver el mayor problema que tienen las redes neuronales ordinarias “el tratamiento de imágenes”.

Trabajan dividiendo y modelando la información en partes más pequeñas, y combinando esta información en las capas más profundas de la red.

Están formadas por una estructura compuesta por tres tipos de capas: capa convolucional, capa de reducción o pooling y capa clasificadora.



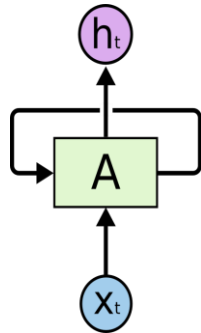
Funcionamiento de una capa convolucional



Max-Pooling aplicada a una imagen

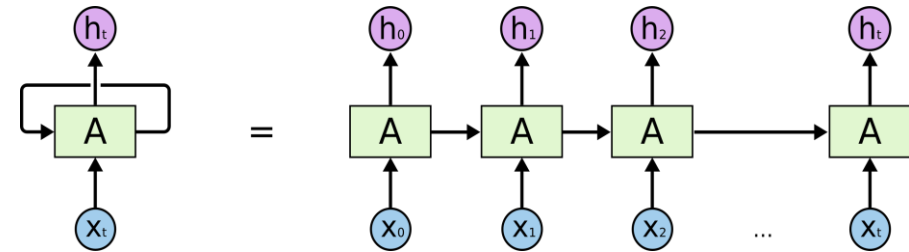
Un poco sobre la estructura de las Redes Neuronales Recurrentes

La característica principal es que son redes con bucles que permiten que la información persista.



Ejemplo básico de una red neuronal recurrente

Una red neuronal recurrente puede ser creada usando múltiples copias de la misma red, pasando el mensaje o la salida al nodo sucesor



Funcionamiento de una capa convolucional

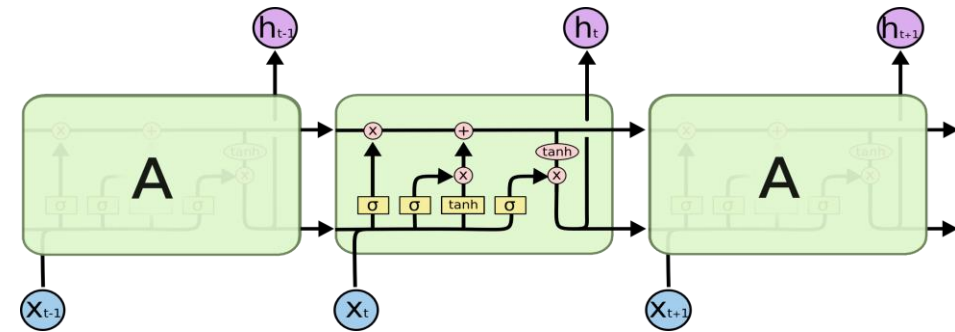
Red Neuronal LSTM (Long Short Term Memory)

Principales características:

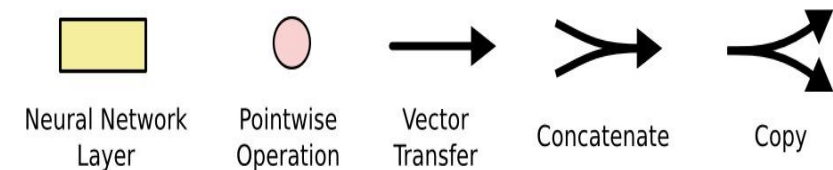
Es una red neuronal recurrente muy especial y muy exitosa.

son capaces de aprender una larga lista de dependencias en largos períodos de tiempo.

El módulo repetidor tiene 4 capas en vez de 1 sola como las redes neuronales estándar.



Estructura del módulo repetidor de una red neuronal LSTM



Notación del módulo repetidor

Módulo 5

Se abordará lo siguiente:

- Actividades prácticas desarrolladas con TensorFlow y Keras.
- Creación e implementación de Chatbots y apps para resolver algunos problemas del entorno educativo
- Accesar al repositorio de Google Drive del curso

[Drive CADI IA TensorFlow](#)



Actividades con TensorFlow y Keras - ChatBots

- Clasificación de actividades educativas en automatizables y no automatizables.
- Calculadora de calificaciones escolares.
- Análisis de calificaciones de estudiantes.
- App para predecir calificaciones finales a partir de hábitos académicos.
- App para practicar derivadas con retroalimentación automática.
- Estimador inteligente de tiempo necesario para estudiar.
- App que permite recomendar recursos educativos personalizados.
- App para el diagnóstico del nivel de comprensión lectora.
- App para analizar sentimientos respecto a opiniones de clases.
- App para recomendar campos de estudio basado en intereses.
- Clasificador automático de respuestas de opción múltiple en Ciencias.
- App para detectar el estilo de aprendizaje basado en respuestas de cuestionarios.
- Clasificador de emociones en frases de estudiantes.
- App para detectar errores comunes en expresiones algebraicas.
- App para entrenar y evaluar vocabulario en inglés mediante reconocimiento de voz.

Actividades con TensorFlow y Keras - ChatBots

- Generador y evaluador de preguntas de Matemáticas con detección automática de respuestas.
- App para reconocimiento y clasificación básica de palabras en inglés con aprendizaje supervisado.
- ChatBot para resolver ecuaciones lineales simples.
- ChatBot para interpretar gráficos estadísticos.
- ChatBot para el cálculo de áreas de figuras geométricas básicas.
- ChatBot de análisis vocacional
- ChatBot para practicar derivadas.
- Planificador inteligente de horarios de clase.
- ChatBot para recomendar recursos educativos personalizados.
- ChatBot sobre estado emocional y psicológico.

Muchas gracias a tod@s por su atención!!!!



“Ser humano es ser **un** humano: una persona específica, con una historia de vida, idiosincrasia y punto de vista”....”Los humanos agregamos valor donde las máquinas no pueden y viceversa”